



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Name: Karan Kumar Pal

PRN: 25070122098

Branch: CSE (25-29)

Semester: 3

Subject: Programming with Java

ASSIGNMENT No.: 10

TITLE: Write a Java program to demonstrate the use of abstract classes and abstract methods

Problem Statement:

Write a Java program to demonstrate the use of abstract classes and abstract methods

Learning Objectives:

To implement abstraction using abstract classes and abstract methods.

Theory:

Abstract classes provide partial implementation of a program and cannot be instantiated directly.

They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

Outcomes:

Abstract classes and abstract methods were successfully used to define a common structure while leaving implementation details to subclasses, as verified through the sample programs — each subclass implemented the abstract method in its own way.

```
abstract class Shape{
    String color;

    Shape(String c){
        this.color = c;
    }

    abstract double area();

    void showColor(){
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape{
    double radius;

    Circle(String c, double r){
        super(c);
        this.radius = r;
    }

    double area(){
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape{
    double length, width;

    Rectangle(String c, double l, double w){
        super(c);
        this.length = l;
        this.width = w;
    }

    double area(){
        return length * width;
    }
}

public class JAVA_EXP_10{
    public static void main(String [] args){
```

```

    Shape circle = new Circle("Green",5);
    circle.showColor();
    System.out.println("Circle Area: " + circle.area());

    System.out.println();

    Shape rectangle = new Rectangle("Yellow",9,5);
    rectangle.showColor();
    System.out.println("Rectanle Area: " + rectangle.area());
}

}
D:\AppData\Roaming\Code\User\workspaceStorage\00113000\
redhat.java\jdt_ws\JAVA_PRACTICALS_eabe1187\bin' 'JAVA_EX
Color: Green
Circle Area: 153.93804002589985

Color: Yellow
Rectangle Area: 45.5
PS C:\Users\abell\OneDrive\Desktop\JAVA_PRACTICALS>

```

Conclusion:

Abstract classes are useful for enforcing a contract that subclasses must follow, while still allowing shared, concrete behaviour to live in the parent class. They support abstraction by hiding implementation details behind a common interface, making programs more flexible, extensible, and easier to maintain as new subclasses are added.

Exercise:

1. Create an abstract Payment class and implement Credit Card and UPI payment classes.

```

abstract class Payment {
    double amount;

    Payment(double amount) {
        this.amount = amount;
    }

    abstract void processPayment();

    void showAmount() {

```

```

        System.out.println("Amount to pay: " + amount);
    }
}

class CreditCardPayment extends Payment {
    String cardNumber;

    CreditCardPayment(double amount, String cardNumber) {
        super(amount);
        this.cardNumber = cardNumber;
    }

    @Override
    void processPayment() {
        showAmount();
        System.out.println("Payment processed via Credit Card ending in "
            + cardNumber.substring(cardNumber.length() - 4));
    }
}

class UPIPayment extends Payment {
    String upiId;

    UPIPayment(double amount, String upiId) {
        super(amount);
        this.upiId = upiId;
    }

    @Override
    void processPayment() {
        showAmount();
        System.out.println("Payment processed via UPI ID: " + upiId);
    }
}

public class JAVA_EXP_10 {
    public static void main(String[] args) {
        Payment payment1 = new CreditCardPayment(1850.0, "4111222233334444");
        payment1.processPayment();

        System.out.println();

        Payment payment2 = new UPIPayment(980.0, "abishek@ybl");
        payment2.processPayment();
    }
}

```

```
in' 'JAVA_EXP_10'
```

```
Amount to pay: 1850.75
```

```
Payment processed via Credit Card ending in 4444
```

```
Amount to pay: 980.50
```

```
Payment processed via UPI ID: abishek@ybl
```

```
PS C:\Users\abishek\OneDrive\Desktop\JAVA_PRACTICALS>
```

2. Create an abstract FoodOrder class with an abstract method calculateBill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.

```
abstract class FoodOrder {
    String customerName;
    double baseAmount;

    FoodOrder(String customerName, double baseAmount) {
        this.customerName = customerName;
        this.baseAmount = baseAmount;
    }

    // abstract method - each order type calculates its bill differently
    abstract double calculateBill();

    void displayOrder() {
        System.out.println("Customer: " + customerName);
        System.out.println("Base Amount: " + baseAmount);
        System.out.println("Total Bill: " + calculateBill());
    }
}

class DineInOrder extends FoodOrder {
    static final double SERVICE_CHARGE_RATE = 0.10; // 10% service charge
    static final double GST_RATE = 0.05;           // 5% GST

    DineInOrder(String customerName, double baseAmount) {
        super(customerName, baseAmount);
    }

    @Override
    double calculateBill() {
        double serviceCharge = baseAmount * SERVICE_CHARGE_RATE;
        double gst = baseAmount * GST_RATE;
        return baseAmount + serviceCharge + gst;
    }
}

class TakeAwayOrder extends FoodOrder {
    static final double PACKAGING_CHARGE = 20.0; // flat packaging fee
    static final double GST_RATE = 0.05;        // 5% GST

    TakeAwayOrder(String customerName, double baseAmount) {
        super(customerName, baseAmount);
    }
}
```

```

@Override
double calculateBill() {
    double gst = baseAmount * GST_RATE;
    return baseAmount + PACKAGING_CHARGE + gst;
}
}

public class JAVA_EXP_10 {
    public static void main(String[] args) {
        FoodOrder order1 = new DineInOrder("Abishek Yadav",
        1250.0); System.out.println("---- Dine-In Order ");
        order1.displayOrder();

        System.out.println();

        FoodOrder order2 = new TakeAwayOrder("Sneha Patil", 750.0);
        System.out.println("---- Take-Away Order ---- ");
        order2.displayOrder();
    }
}

```

```

PS C:\Users\abishek\OneDrive\Desktop\JAVA_PRACTICALS_eabe1187\bin> 'JAVA_EXP_10'

---- Dine-In Order ----
Customer: Abishek Yadav
Base Amount: 1250.0
Total Bill: 1375.0

---- Take-Away Order ----
Customer: Priya Sharma
Base Amount: 750.0
Total Bill: 825.0
PS C:\Users\abishek\OneDrive\Desktop\JAVA_PRACTICALS>

```